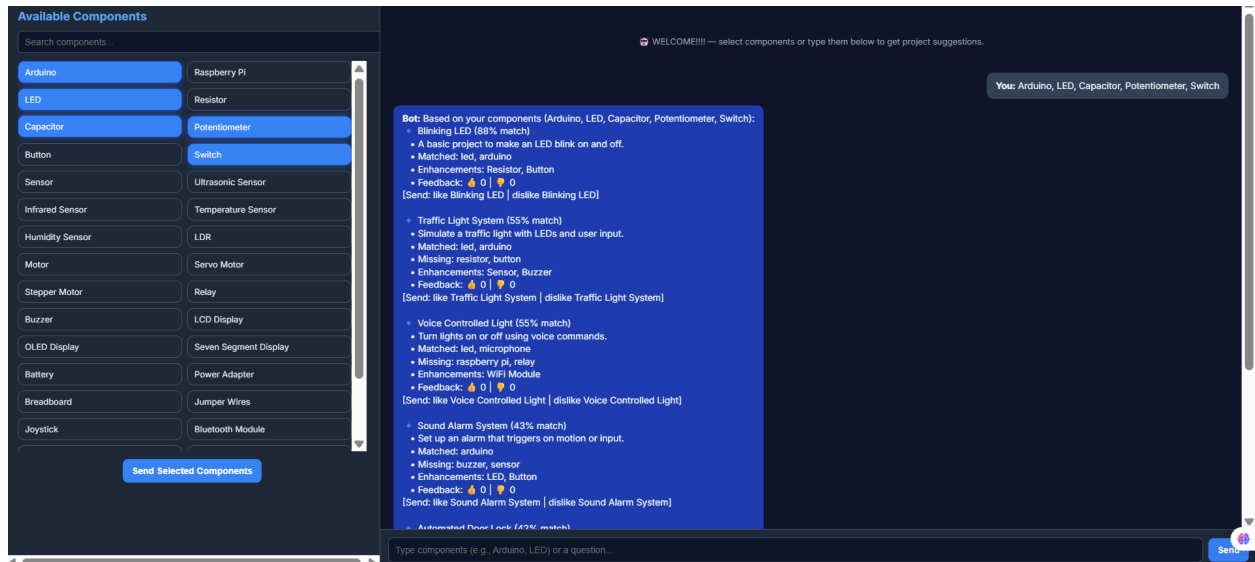


AI-POWERED PROJECT RECOMMENDER SYSTEM



PANCHAMI DINESH

27.10.2025

1. Introduction

The **AI Project Recommender System** is an intelligent web-based application that suggests suitable projects based on the components or hardware items provided by the user.

For example, if a user says, “*I have a Raspberry Pi and a temperature sensor,*” the system recommends relevant projects such as *Weather Station* or *Smart Home Sensor* with match percentages and missing components.

The system combines **rule-based logic** (for direct component matching) with **semantic understanding** using machine learning embeddings.

It also learns over time from user feedback, improving its future recommendations.

2. Objectives

- To build an AI-driven system that understands user input in natural language.
 - To suggest the most relevant projects based on components or keywords.
 - To integrate semantic similarity using **Sentence-BERT embeddings**.
 - To allow feedback-based learning for adaptive ranking.
 - To provide a clean and interactive user interface using **Flask + SocketIO**.
-

3. System Architecture

The architecture has three main parts:

(a) Frontend (UI)

- Built with **HTML, CSS, and JavaScript**.
- Real-time message exchange is handled through **Flask-SocketIO**.

- Users can type queries, view ranked project suggestions, and send feedback (like/dislike).
- Dynamic updates without page reload.

(b) Backend (AI Engine)

- Developed using **Python Flask**.
- Uses **SentenceTransformer** ('all-MiniLM-L6-v2') to generate vector embeddings for each project and user query.
- Uses **spaCy** or **regex-based fallback** to extract key components from natural language.
- Performs **fuzzy matching** via the **rapidfuzz** library to match component names even with typos or variations.
- Combines logical component matching with semantic similarity for accurate ranking.
- Updates user feedback persistently in **feedback.json**.

(c) Knowledge Base

- Stored in a structured **data.json** file.
- Contains:
 - A list of known components.
 - Relationships between components (synonyms, related terms).
 - Predefined projects with required and optional components and descriptions.

4. Workflow

Step 1: Input

User enters a message such as:

“I have a Raspberry Pi and temperature sensor, what can I build?”

Step 2: Component Extraction

The system identifies components from the text:

- “raspberry pi”
- “temperature sensor”

If spaCy is available, it uses **noun chunk extraction**; otherwise, it uses regex tokenization.

Step 3: Component Expansion

Each component is expanded using `component_relations` from `data.json`.
For example:

- “temperature sensor” → [“sensor”, “thermistor”]
- “raspberry pi” → [“microcontroller”, “WiFi module”]

Step 4: Semantic Matching

Every project description from `data.json` is embedded into a vector using **Sentence-BERT**.

The user input is also embedded.

Then, **cosine similarity** is computed to find semantically similar projects.

Step 5: Combined Scoring

A final score is calculated as:

$$1. \text{ combined_score} = 0.65 * \text{logical_match} + 0.35 * \text{semantic_similarity}$$

This ensures that both *exact component matches* and *semantic understanding* are considered.

Step 6: Feedback Learning

Each project's score is slightly boosted or penalized based on previous likes/dislikes from `feedback.json`.

$$2. \text{ feedback_factor} = (\text{likes} - \text{dislikes}) * 0.03$$

Step 7: Response Display

Top project suggestions are displayed in the chat window with:

- Match percentage
- Description
- Matched and missing components
- Suggested enhancements
- Feedback buttons (like/dislike)

Example output:

3. Based on your components (raspberry pi, temperature sensor):
4. ♦ Weather Station (78% match)
5. • Monitors temperature and humidity using sensors.
6. • Matched: raspberry pi

7. • Missing: temperature sensor, lcd display
8. • Enhancements: WiFi Module, Buzzer
9. • Feedback: 👍 2 | 👎 0
10. [Send: like Weather Station | dislike Weather Station]

5. Technologies Used

Component	Technology / Library
Frontend	HTML, CSS, JavaScript
Backend	Python Flask
Real-time Communication	Flask-SocketIO
AI Embedding Model	SentenceTransformer (all-MiniLM-L6-v2)
NLP Engine	spaCy (en_core_web_sm)
Fuzzy Matching	rapidfuzz
Data Storage	JSON (data.json, feedback.json)

6. Files and Structure

11. AI_Project_Recommender/
12. |
13. |— app.py # Main backend application
14. |— data.json # Knowledge base of components and projects
15. |— feedback.json # User feedback data (auto-created)

- 16. |— templates/
 - 17. | |— index.html # Frontend UI
 - 18. |— static/
 - 19. | |— style.css # Custom styling (optional)
 - 20. | |— script.js # SocketIO interaction logic
 - 21. |— README.md # Documentation
-

7. Features

- ✓ Understands natural language queries.
 - ✓ Handles typos and variations (fuzzy matching).
 - ✓ Ranks projects by relevance.
 - ✓ Displays missing and matched components.
 - ✓ Learns from feedback (adaptive ranking).
 - ✓ Works offline once model is loaded.
 - ✓ Simple and attractive web interface.
-

8. Sample Queries You Can Try

- “I have a Raspberry Pi and a temperature sensor, what can I build?”
 - “What can I make using Arduino and LEDs?”
 - “I have a relay, motor, and battery.”
 - “Projects related to sensors and IoT?”
 - “Show me something with ultrasonic and servo motor.”
 - “What are the components needed for Smart Home Sensor?”
-

9. Output Example

- 22. Based on your components (arduino, led, button):
 - 23. ♦ Traffic Light System (86% match)
 - 24. • Simulate a traffic light using LEDs and button control.
 - 25. • Matched: arduino, led, button
 - 26. • Missing: resistor
 - 27. • Enhancements: Sensor, Buzzer
 - 28. • Feedback: 👍 1 | 👎 0
 - 29. [Send: like Traffic Light System | dislike Traffic Light System]
-

10. Future Enhancements

- **Voice input support** for hands-free interaction.
- **Integration with Gemini or Llama** to generate natural explanations.
- **Auto feedback learning** to adapt from user selections.
- **React.js or Flutter frontend** for mobile-friendly UI.
- **Database integration** (SQLite or Firebase) instead of JSON.

11. Conclusion

This project demonstrates how **Natural Language Processing (NLP)** and **semantic similarity modeling** can be applied to an intelligent recommendation system.

It bridges human-like understanding with structured data, allowing users to discover projects based on their available components intuitively.

The system can easily be expanded for educational institutions, maker spaces, or IoT learning platforms.

